C++ Programming: From Problem Analysis to Program Design, Third Edition

Chapter 9: Arrays and Strings

Objectives

In this chapter you will:

- Learn about arrays
- Explore how to declare and manipulate data into arrays
- Understand the meaning of “array index out of bounds”
- Become familiar with the restrictions on array processing
- Discover how to pass an array as a parameter to a function

Objectives (continued)

- Learn about C-strings
- Examine the use of string functions to process C-strings
- Discover how to input data into—and output data from—a C-string
- Learn about parallel arrays
- Discover how to manipulate data in a two-dimensional array

Data Types

- A data type is called simple if variables of that type can store only one value at a time
- A structured data type is one in which each data item is a collection of other data items

Arrays

- Array - a collection of a fixed number of components wherein all of the components have the same data type
- One-dimensional array - an array in which the components are arranged in a list form
- The general form of declaring a one-dimensional array is:

```
dataType arrayName[intExp];
```

where `intExp` is any expression that evaluates to a positive integer

Declaring an array

- The statement

  ```
  int num[5];
  ```

  declares an array `num` of `5` components of the type `int`
- The components are `num[0]`, `num[1]`, `num[2]`, `num[3]`, and `num[4]`

FIGURE 9-1 Array num

Accessing Array Components

- The general form (syntax) of accessing an array component is:

  `arrayName[indexExp]`

  where `indexExp`, called **index**, is any expression whose value is a nonnegative integer
- Index value specifies the position of the component in the array
- The `[]` operator is called the **array subscripting operator**
- The array index always starts at `0`

```
int list[10];
```

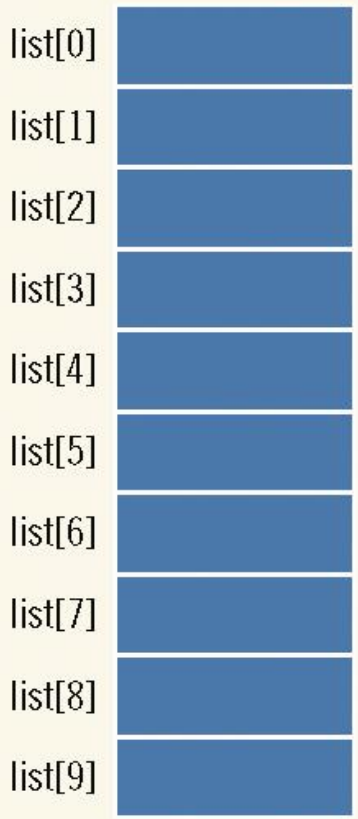

FIGURE 9-2 Array `list`

```
list[5] = 34;
```

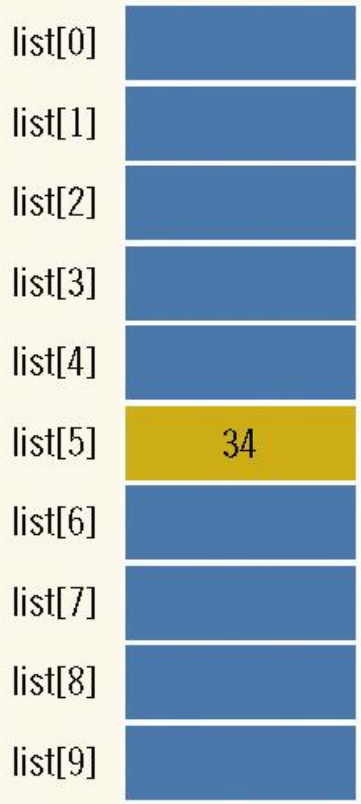

FIGURE 9-3 Array `list` after execution of the statement `list[5]= 34;`

```
list[3] = 10;
list[6] = 35;
list[5] = list[3] + list[6];
```

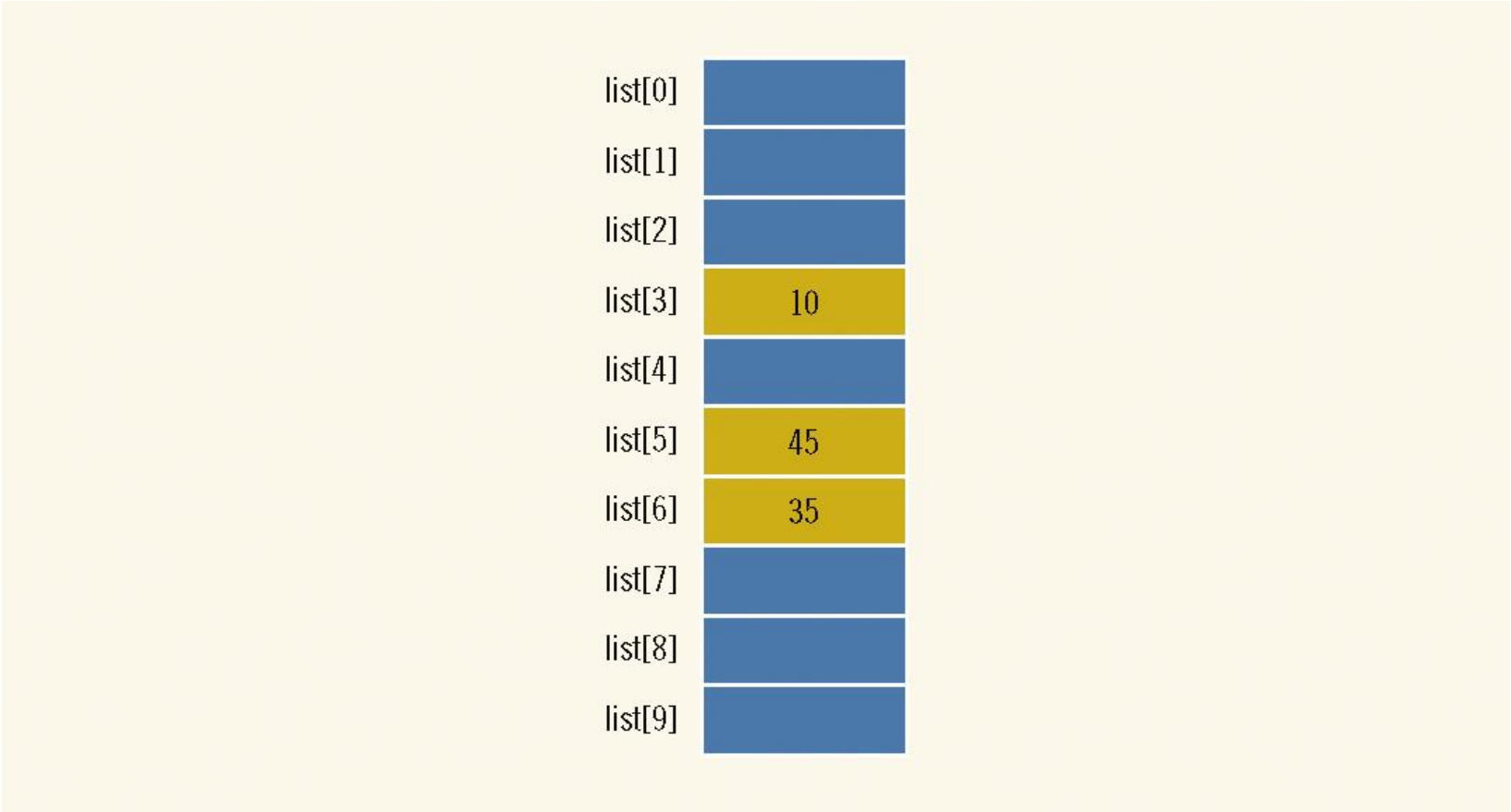

FIGURE 9-4 Array `list` after execution of the statements `list[3]= 10;`, `list[6]= 35;`, and `list[5] = list[3] + list[6];`

EXAMPLE 9-2

You can also declare arrays as follows:

```
const int ARRAY_SIZE = 10;
int list[ARRAY_SIZE];
```

That is, you can first declare a named constant and then use the value of the named constant to declare an array and specify its size.

NOTE When you declare an array, its size must be known. For example, you cannot do the following:

```
int arraySize;                                  //Line 1

cout << "Enter the size of the array: ";  //Line 2
cin >> arraySize;                               //Line 3
cout << endl;                                   //Line 4

int list[arraySize];                            //Line 5; not allowed
```

Processing One-Dimensional Arrays

- Some basic operations performed on a one-dimensional array are:
  - Initialize
  - Input data
  - Output data stored in an array
  - Find the largest and/or smallest element
- Each operation requires ability to step through the elements of the array
- Easily accomplished by a loop

Accessing Array Components

- Consider the declaration

```
int list[100];  //list is an array
                //of the size 100
int i;
```

- This `for` loop steps-through each element of the array list starting at the first element

```
for (i = 0; i < 100; i++) //Line 1
    //process list[i]     //Line 2
```

Accessing Array Components (continued)

- If processing list requires inputting data into list
  - the statement in Line 2 takes the from of an input statement, such as the `cin` statement

```
for (i = 0; i < 100; i++)      //Line 1
    cin >> list[i];
```

EXAMPLE 9-3

```
double sales[10];
int index;
double largestSale, sum, average;
```

Initializing an array:

```
for (index = 0; index < 10; index++)
    sales[index] = 0.0;
```

Reading data into an array:

```
for (index = 0; index < 10; index++)
    cin >> sales[index];
```

Printing an array:

```
for (index = 0; index < 10; index++)
    cout << sales[index] << " ";
```

Finding the sum and average of an array:

```
sum = 0;
for (index = 0; index < 10; index++)
     sum = sum + sales[index];

average = sum / 10;
```

Largest element in the array:

```
maxIndex = 0;
for (index = 1; index < 10; index++)
     if (sales[maxIndex] < sales[index])
          maxIndex = index;
largestSale = sales[maxIndex];
```

Array Index Out of Bounds

- If we have the statements:

```
double num[10];

int i;
```

- The component `num[i]` is a valid index if `i = 0, 1, 2, 3, 4, 5, 6, 7, 8,` or `9`
- The index of an array is in bounds if the `index >=0` and the `index <= ARRAY_SIZE-1`

Array Index Out of Bounds (continued)

- If either the `index < 0` or the `index > ARRAY_SIZE-1`
  - then we say that the `index` is out of bounds
- There is no guard against indices that are out of bounds
  - C++ does not check if the index value is within range

Array Initialization

- As with simple variables
  - Arrays can be initialized while they are being declared
- When initializing arrays while declaring them
  - Not necessary to specify the size of the array
- Size of array is determined by the number of initial values in the braces
- For example:

```
double sales[] = {12.25, 32.50, 16.90, 23,
                  45.68};
```

Partial Initialization

- The statement

```
int list[10] = {0};
```

  declares `list` to be an array of `10` components and initializes all components to zero

- The statement

```
int list[10] = {8, 5, 12};
```

  declares `list` to be an array of `10` components, initializes `list[0]` to `8`, `list[1]` to `5`, `list[2]` to `12` and all other components are initialized to `0`

Partial Initialization (continued)

- The statement

```
int list[] = {5, 6, 3};
```

  declares `list` to be an array of `3` components and initializes `list[0]` to `5`, `list[1]` to `6`, and `list[2]` to `3`

- The statement

```
int list[25]= {4, 7};
```

  declares `list` to be an array of `25` components
  - The first two components are initialized to `4` and `7` respectively
  - All other components are initialized to `0`

Restrictions on Array Processing

```
int myList[5] = {0, 4, 8, 12, 16};  //Line 1
int yourList[5];  //Line 2
```

Assignment does not work with arrays

```
yourList = myList;  //illegal
```

In order to copy one array into another array we must copy component-wise

```
for (int index = 0; index < 5; index ++)
    yourList[index] = myList[index];
```

Restrictions on Array Processing (continued)

```
cin >> yourList; //illegal
```

```
for (int index = 0; index < 5; index ++)
     cin >> yourList[index];
```

Arrays as Parameters to Functions

- Arrays are passed by reference only
- The symbol & is *not* used when declaring an array as a formal parameter
- The size of the array is usually omitted

Arrays as Parameters to Functions (continued)

- If the size of one-dimensional array is specified when it is declared as a formal parameter
  - It is ignored by the compiler
- The reserved word `const` in the declaration of the formal parameter can prevent the function from changing the actual parameter

EXAMPLE 9-5

```
void funcArrayAsParam(int listOne[], double listTwo[])
{
      .
      .
      .
}
```

Constant Arrays as Formal Parameters

```
void example(int x[], const int y[], int sizeX, int sizeY)
{
      .
      .
      .
}
```

EXAMPLE 9-6

```
    //Function to initialize an int array to 0.
    //The array to be initialized and its size are passed
    //as parameters. The parameter listSize specifies the
    //number of elements to be initialized.
void initializeArray(int list[], int listSize)
{
    int index;

    for (index = 0; index < listSize; index++)
        list[index] = 0;
}

    //Function to read and store the data into an int array.
    //The array to store the data and its size are passed as
    //parameters. The parameter listSize specifies the number
    //of elements to be read.
void fillArray(int list[], int listSize)
{
    int index;

    for (index = 0; index < listSize; index++)
        cin >> list[index];
}
```

```
    //Function to print the elements of an int array.
    //The array to be printed and the number of elements
    //are passed as parameters. The parameter listSize
    //specifies the number of elements to be printed.
void printArray(const int list[], int listSize)
{
    int index;

    for (index = 0; index < listSize; index++)
        cout << list[index] << " ";
}
    //Function to find and return the sum of the
    //elements of an int array. The parameter listSize
    //specifies the number of elements to be added.
int sumArray(const int list[], int listSize)
{
    int index;
    int sum = 0;

    for (index = 0; index < listSize; index++)
        sum = sum + list[index];

    return sum;
}
```

```
    //Function to find and return the index of the first
    //largest element in an int array. The parameter listSize
    //specifies the number of elements in the array.
int indexLargestElement(const int list[], int listSize)
{
    int index;
    int maxIndex = 0; //assume the first element is the largest

    for (index = 1; index < listSize; index++)
        if (list[maxIndex] < list[index])
            maxIndex = index;

    return maxIndex;
}

    //Function to copy one array into another array.
    //The elements of listOne are copied into listTwo.
    //The array listTwo must be at least as large as the
    //number of elements to be copied. The parameter
    //listOneSize specifies the number of elements of
    //listOne to be copied into listTwo.
void copyArray(const int listOne[], int listTwo[],
               int listOneSize)
{
    int index;

    for (index = 0; index < listOneSize; index++)
        listTwo[index] = listOne[index];
}
```

Base Address of an Array

- The base address of an array is the address, or memory location of the first array component
- If `list` is a one-dimensional array
  - base address of list is the address of the component `list[0]`
- When we pass an array as a parameter
  - base address of the actual array is passed to the formal parameter
- Functions cannot return a value of the type array

C Strings (Character Arrays)

- Character array - an array whose components are of type char
- String - a sequence of zero or more characters enclosed in double quote marks
- C stings are null terminated (‘\0’)
- The last character in a string is the null character

C Strings (Character Arrays) (continued)

- There is a difference between `'A'` and `"A"`
  - `'A'` is the character `A`
  - `"A"` is the string `A`
- Because strings are null terminated, `"A"` represents two characters, `'A'` and `'\0'`
- Similarly, `"Hello"` contains six characters, `'H'`, `'e'`, `'l'`, `'l'`, `'o'`, and `'\0'`

C Strings (Character Arrays) (continued)

- Consider the statement

```
char name[16];
```

- Because C strings are null terminated and name has sixteen components
  - The largest string that can be stored in name is `15`
- If you store a string of length, say 10 in `name`
  - The first 11 components of `name` are used and the last 5 are left unused

C Strings (Character Arrays) (continued)

- The statement

```
char name[16] = "John";
```

  declares a string variable `name` of length 16 and stores `"John"` in it

- The statement

```
char name[] = "John";
```

  declares a string variable name of length 5 and stores `"John"` in it

TABLE 9-1 `strcpy`, `strcmp`, and `strlen` functions

Function	Effect
`strcpy(s1, s2)`	Copies the string `s2` into the string variable `s1` The length of `s1` should be at least as large as `s2`
`strcmp(s1, s2)`	Returns a value < `0` if `s1` is less than `s2` Returns `0` if `s1` and `s2` are the same Returns a value > `0` if `s1` is greater than `s2`
`strlen(s)`	Returns the length of the string `s`, excluding the null character

String Comparison

- C-strings are compared character by character using the collating sequence of the system
- If we are using the ASCII character set
  1. The string `"Air"` is smaller than the string `"Boat"`
  2. The string `"Air"` is smaller than the string `"An"`
  3. The string `"Bill"` is smaller than the string `"Billy"`
  4. The string `"Hello"` is smaller than `"hello"`

EXAMPLE 9-8

```
char studentName[21];
char myname[16];
char yourname[16];
```

Statement	Effect
`strcpy(myname, "John Robinson");`	`Myname = "John Robinson"`
`strlen("John Robinson");`	Returns 13, the length of the string `"John Robinson"`
`int len;` `len = strlen("Sunny Day");`	Stores 9 into `len`
`strcpy(yourname, "Lisa Miller");` `strcpy(studentName, yourname);`	`yourname = "Lisa Miller"` `studentName = "Lisa Miller"`
`strcmp("Bill", "Lisa");`	Returns a value < 0

`strcpy(yourname, "Kathy Brown");`
`strcpy(myname, "Mark G. Clark");`
`strcmp(myname, yourname);`
	`yourname = "Kathy Brown"`
	`myname = "Mark G. Clark"`
	Returns a value > 0

Reading and Writing Strings

- String Input
  - Aggregate operations are allowed for string input
  - `cin >> name;` stores the next input string in name
  - Strings that contain blanks cannot be read using the extraction operator `>>`

Reading and Writing Strings (continued)

- To read strings with blanks, use the `get` function with an input stream variable, such as `cin`:

```
cin.get(str, m+1);
```

  - Stores the next `m` characters into `str` but the newline character is not stored in `str`
  - If the input string has fewer than `m` characters, the reading stops at the newline character

Reading and Writing Strings (continued)

- String Output
  - The statement `cout << name;` outputs the content of name on the screen
  - The insertion operator `<<` continues to write the contents of name until it finds the null character
  - If name does not contain the null character, then we will see strange output: `<<` continues to output data from memory adjacent to name until `'\0'` is found

Input/Output Files

- C++ strings are not null terminated
- Variables of type string can be used to read and store the names of input/output files
- The argument to the function open must be a null terminated string (a C-string)
- If we use a variable of type string to read the name of an input/output file and then use this variable to open a file
  - The value of the variable must first be converted to a C-string (null-terminated string)

Input/Output Files (continued)

- The header file string contains the function `c_str`
  - Converts a value of the type string to a null-terminated character array
- The syntax to use the function `c_str` is:

```
strVar.c_str()
```

where `strVar` is a variable of type `string`

Parallel Arrays

- Two (or more) arrays are called parallel if their corresponding components hold related information
- For example:

```
int studentId[50];

char courseGrade[50];
```

Two-Dimensional Arrays

- Two-dimensional Array: a collection of a fixed number of components arranged in two dimensions
  - All components are of the same type
- The syntax for declaring a two-dimensional array is:

```
dataType arrayName[intexp1][intexp2];
```

where `intexp1` and `intexp2` are expressions yielding positive integer values

Two-Dimensional Arrays (continued)

- The two expressions `intexp1` and `intexp2` specify the number of rows and the number of columns, respectively, in the array
- Two-dimensional arrays are sometimes called matrices or tables

```
double sales[10][5];
```

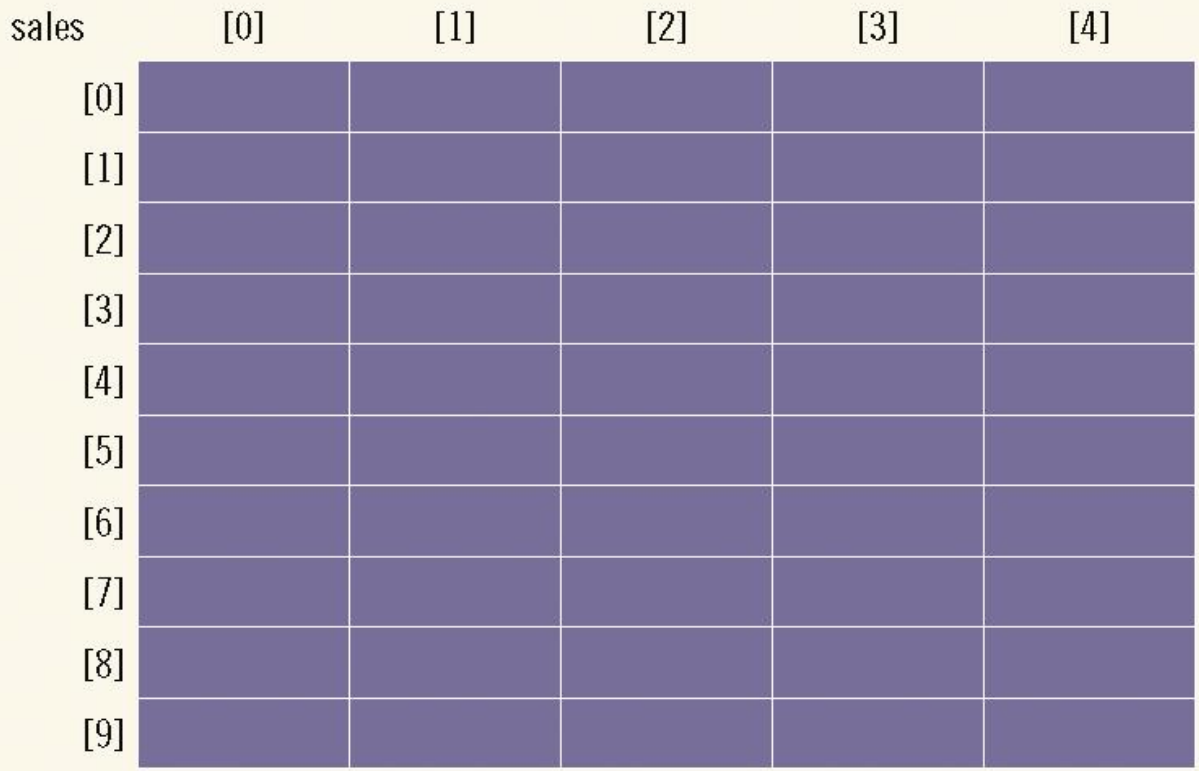

FIGURE 9-8 Two-dimensional array `sales`

Accessing Array Components

- The syntax to access a component of a two-dimensional array is:

  `arrayName[indexexp1][indexexp2]`

  where `indexexp1` and `indexexp2` are expressions yielding nonnegative integer values
- `indexexp1` specifies the row position and `indexexp2` specifies the column position

```
sales[5][3] = 25.75;
```

FIGURE 9-12 `inStock[FORD][WHITE]`

Initialization

- Like one-dimensional arrays
  - Two-dimensional arrays can be initialized when they are declared
- To initialize a two-dimensional array when it is declared
  1. Elements of each row are enclosed within braces and separated by commas
  2. All rows are enclosed within braces
  3. For number arrays, if all components of a row are not specified, the unspecified components are initialized to zero

Processing Two-Dimensional Arrays

- A two-dimensional array can be processed in three different ways:
  1. Process the entire array
  2. Process a particular row of the array, called row processing
  3. Process a particular column of the array, called column processing

Processing Two-Dimensional Arrays (continued)

- Each row and each column of a two-dimensional array is a one-dimensional array
- When processing a particular row or column of a two-dimensional array
  - We use algorithms similar to processing one-dimensional arrays

```
const int NUMBER_OF_ROWS = 7;     //This can be set to any number.
const int NUMBER_OF_COLUMNS = 6; //This can be set to any number.

int matrix[NUMBER_OF_ROWS][NUMBER_OF_COLUMNS];
int row;
int col;
int sum;
int largest;
int temp;
```

Initialization

```
for (row = 0; row < NUMBER_OF_ROWS; row++)
    for (col = 0; col < NUMBER_OF_COLUMNS; col++)
        matrix[row][col] = 0;
```

Print

```
for (row = 0; row < NUMBER_OF_ROWS; row++)
{
    for (col = 0; col < NUMBER_OF_COLUMNS; col++)
        cout << setw(5) << matrix[row][col] << " ";

    cout << endl;
}
```

Input

```
for (row = 0; row < NUMBER_OF_ROWS; row++)
    for (col = 0; col < NUMBER_OF_COLUMNS; col++)
        cin >> matrix[row][col];
```

Sum by Row

```
  //Sum of each individual row
for (row = 0; row < NUMBER_OF_ROWS; row++)
{
    sum = 0;
    for (col = 0; col < NUMBER_OF_COLUMNS; col++)
        sum = sum + matrix[row][col];

    cout << "Sum of row " << row + 1 << " = " << sum << endl;
}
```

Sum by Column

```
  //Sum of each individual column
for (col = 0; col < NUMBER_OF_COLUMNS; col++)
{
    sum = 0;
    for (row = 0; row < NUMBER_OF_ROWS; row++)
        sum = sum + matrix[row][col];

    cout << "Sum of column " << col + 1 << " = " << sum
         << endl;
}
```

Largest Element in Each Row and Each Column

```
  //Largest element in each row
for (row = 0; row < NUMBER_OF_ROWS; row++)
{
    largest = matrix[row][0]; //Assume that the first element
                              //of the row is the largest.
    for (col = 1; col < NUMBER_OF_COLUMNS; col++)
        if (largest < matrix[row][col])
            largest = matrix[row][col];

    cout << "The largest element in row " << row + 1 << " = "
         << largest << endl;
}

  //Largest element in each column
for (col = 0; col < NUMBER_OF_COLUMNS; col++)
{
    largest = matrix[0][col]; //Assume that the first element
                              //of the column is the largest.
    for (row = 1; row < NUMBER_OF_ROWS; row++)
        if (largest < matrix[row][col])
            largest = matrix[row][col];

    cout << "The largest element in column " << col + 1
         << " = " << largest << endl;
}
```

Passing Two-Dimensional Arrays as Parameters to Functions

- Two-dimensional arrays can be passed as parameters to a function
- By default, arrays are passed by reference
- The base address, that is, the address of the first component of the actual parameter is passed to the formal parameter

Two-Dimensional Arrays

- Two-dimensional arrays are stored in row order
  - The first row is stored first, followed by the second row, followed by the third row and so on
- When declaring a two-dimensional array as a formal parameter
  - Can omit size of first dimension, but not the second
- Number of columns must be specified

Summary

- An array is a structured data type with a fixed number of components
  - Every component is of the same type
  - Components are accessed using their relative positions in the array
- Elements of a one-dimensional array are arranged in the form of a list
- An array index can be any expression that evaluates to a non-negative integer
- The value of the index must always be less than the size of the array

Summary (continued)

- The base address of an array is the address of the first array component
- In a function call statement, when passing an array as an actual parameter, you use only its name
- As parameters to functions, arrays are passed by reference only
- A function cannot return a value of the type array
- In C++, C-strings are null terminated and are stored in character arrays

Summary (continued)

- Commonly used C-string manipulation functions include: `strcpy`, `strcmp`, and `strlen`
- Parallel arrays are used to hold related information
- In a two-dimensional array, the elements are arranged in a table form

Summary

- To access an element of a two-dimensional array, you need a pair of indices: one for the row position and one for the column position
- In row processing, a two-dimensional array is processed one row at a time
- In column processing, a two-dimensional array is processed one column at a time